

Free-to-play Versus Paid Games' Player Retention on Steam

MATH 189 SP 26

0 Info

Group 41 Team:

- Ashley Vo
- Ana Guzman
- Julian Zaw
- Kangnan Ye
- Tony Tran

Acknowledgements:

- ChatGPT and Claude were used for assistance in our project.
-

1 Introduction

1.1 Background

Steam is a platform for personal computer (PC) games, offering a storefront, streaming service, social networking, and many other features. More specifically, Steam offers a vast platform for game developers to share and sell their games.

There are two primary pricing models that we're interested in analyzing: free-to-play (often abbreviated as F2P) and paid. A free-to-play game is one that does not require an upfront cost to download and play the game. In other words, it can be thought of as costing "\$0" as the name suggests. A paid game is one that requires a cost to download and play.

1.2 Research Question

For games released before 2025, do the top 50 free-to-play games have stronger player retention compared to top 50 paid games in the long-term through their review and player count trends on Steam?

Here, we define "top" games as ones through their estimated ownership, total review count, and launch-period player count. We will not be paying special attention to games that have changed pricing models e.g. paid to free-to-play. There are more details on that in our analysis and limitations section.

1.3 Expectations

We expect free-to-play games to have larger initial player counts and possibly stronger long-term retention due to their accessibility in comparison. However, the question does not account for game genres, multiplayer or live-service status, etc.

1.4 Motivations

From a developer and business perspective, this is a choice that must be made in the industry. Developers may be interested in making a game more accessible by making it free-to-play, but that brings up a question of where and how developers would recuperate development costs if an upfront cost is not required. Player retention is something we're interested in investigating as a sustainable player base could contribute to a game's popularity and possibly support the developers financially if microtransactions are present.

1.5 Similar Work

What may additionally motivate developers is revenue. Newzoo found free-to-play games and their engagement found more efficient revenue, especially on PC platforms. While our analysis will be looking at player retention, having players is linked to revenue. Of course, there are other factors to consider like having some subscription models, microtransactions, and other purchasable content for players. Another study by Wendi Zhang looked at mobile games and their retention methods. More specifically, one of the games they looked at was Clash Royale, a popular free-to-play game. There is great mention of monetization methods but also game design that encourages monetary spending. In short, it is important for us to note that merely free-to-play and paid status alone are not the only indicators of retention success.

- [1] <https://newzoo.com/resources/blog/free-to-play-monetization-pc-vs-console-western-markets>
 - [2] <https://doi.org/10.17760/D20420747>
-

2 Data

There are a couple of sources where our data was obtained from:

- Steam
- SteamCharts
- Kaggle

On Kaggle, our team worked with "Steam Dataset 2025" by Artemiy Ermilov (<https://www.kaggle.com/datasets/artermiloff/steam-games-dataset>). This dataset contains a variety of useful attributes for us like price, genres, estimated ownership, etc. This is the set we used to determine what was a top 50 in each category.

Steam (<https://store.steampowered.com/>) is the official site. Much in our interest, individual game pages have an option to view a chart of the game's historical review data. Thus, we scraped the website for this data after receiving the list of the top 50 games in each category.

SteamCharts (<https://steamcharts.com/>) provides average and peak player count data per month since a game's release. This website was scraped once we gathered the top 50 games from each category. However, due to a data availability error, we were only able to obtain 47 of the 50 paid games' player data. Please see the limitations section for more details.

3 EDA

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.formula.api as smf

from pathlib import Path
from scipy import stats
```

There are some custom functions that we have created for analysis. To prevent their length from taking away from the analysis, they will instead be imported as a module.

```
In [2]: %load_ext autoreload
%autoreload 2
from modules import eda, stat_inf
```

3.1 Data Inspecting

First, we'll examine and explore the data of the top 50 free and paid games list.

```
In [3]: df_free = pd.read_csv('data/top50_free_games_list.csv')
df_paid = pd.read_csv('data/top50_paid_games_list.csv')

print(df_free.head(), df_paid.head())
```

	AppID	Game Title	Release Date	Estimated Owners	\
0	730	Counter-Strike 2	2012-08-21	100M+	
1	570	Dota 2	2013-07-09	100M+	
2	578080	PUBG: BATTLEGROUNDS	2017-12-21	50M - 100M	
3	1172470	Apex Legends	2020-11-04	50M - 100M	
4	230410	Warframe	2013-03-25	50M - 100M	

	Peak Concurrent Players (24h)
0	1415913
1	632090
2	798913
3	277066
4	75491

	AppID	Game Title	Release Date	Base Price	Estimated Owners	\
0	271590	Grand Theft Auto V Legacy	2015-04-13	\$29.99	50M - 100M	
1	105600	Terraria	2011-05-16	\$9.99	20M - 50M	
2	292030	The Witcher 3: Wild Hunt	2015-05-18	\$39.99	20M - 50M	
3	4000	Garry's Mod	2006-11-29	\$9.99	20M - 50M	
4	550	Left 4 Dead 2	2009-11-17	\$9.99	20M - 50M	

	Peak Concurrent Players (24h)
0	88110
1	37858
2	17925
3	30201
4	18706

```
In [4]: df_free.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   AppID                                50 non-null    int64
 1   Game Title                           50 non-null    str
 2   Release Date                         50 non-null    str
 3   Estimated Owners                     50 non-null    str
 4   Peak Concurrent Players (24h)       50 non-null    int64
dtypes: int64(2), str(3)
memory usage: 2.1 KB
```

By looking at the data types, we observe estimated ownership is a string, so we will need to modify it later to work with it nicely. We can expect this is the same of the paid games

dataset as they come from the same source as seen above the `.info()`.

Lastly, let's ensure there are no nulls in either dataset.

```
In [5]: print('Number of non-nulls in free games dataset:\n', df_free.notna().sum(), '\n')
        print('Number of non-nulls in paid games dataset:\n', df_paid.notna().sum())
```

Number of non-nulls in free games dataset:

AppID	50
Game Title	50
Release Date	50
Estimated Owners	50
Peak Concurrent Players (24h)	50

dtype: int64

Number of non-nulls in paid games dataset:

AppID	50
Game Title	50
Release Date	50
Base Price	50
Estimated Owners	50
Peak Concurrent Players (24h)	50

dtype: int64

Now, let's examine the player data.

```
In [6]: df_free_players = pd.read_csv('data/top50_free_games_player_data.csv')
        df_paid_players = pd.read_csv('data/top50_paid_games_player_data.csv')

        print(df_free_players.head(), df_paid_players.head())
```

	game_id	game_name	year_month	peak_player_count \
0	730	Counter-Strike	2026-04	1564830
1	730	Counter-Strike	2026-03	1717624
2	730	Counter-Strike	2026-02	1627561
3	730	Counter-Strike	2026-01	1654355
4	730	Counter-Strike	2025-12	1620137

	average_player_count
0	974785.71
1	1069173.12
2	1084094.59
3	1070443.78
4	991176.89

	game_id	game_name	year_month	peak_pla
0	271590	Grand Theft Auto V	Legacy 2026-04	108812
1	271590	Grand Theft Auto V	Legacy 2026-03	127424
2	271590	Grand Theft Auto V	Legacy 2026-02	127027
3	271590	Grand Theft Auto V	Legacy 2026-01	129542
4	271590	Grand Theft Auto V	Legacy 2025-12	136591

	average_player_count
0	61797.66
1	69770.16
2	77909.39
3	80873.87
4	81832.19

```
In [7]: df_free_players.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5015 entries, 0 to 5014
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   game_id                5015 non-null   int64
1   game_name              5015 non-null   str
2   year_month             5015 non-null   str
3   peak_player_count      5015 non-null   int64
4   average_player_count   5015 non-null   float64
dtypes: float64(1), int64(2), str(2)
memory usage: 196.0 KB
```

The data types are as expected, and like before, since the data for the paid games player data comes from the same source, we can safely assume both have the same data types.

3.2 Average player count trends

Now, let's observe the general trends of player data, starting with the distribution of player counts.

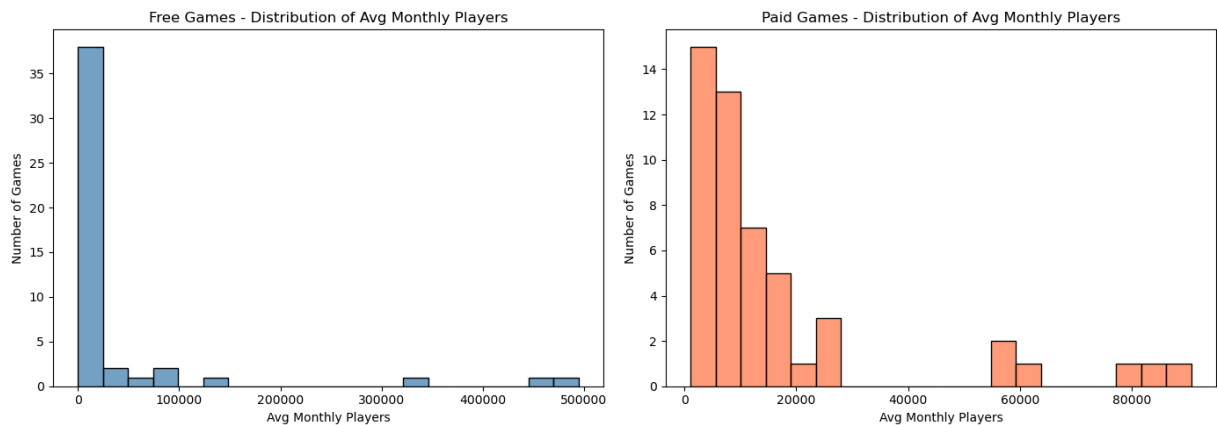
```
In [8]: df_free_agg = df_free_players.groupby('game_id')['average_player_count'].mean()
df_paid_agg = df_paid_players.groupby('game_id')['average_player_count'].mean()
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(df_free_agg, bins=20, ax=axes[0], color='steelblue')
axes[0].set_title('Free Games - Distribution of Avg Monthly Players')
axes[0].set_xlabel('Avg Monthly Players')
axes[0].set_ylabel('Number of Games')

sns.histplot(df_paid_agg, bins=20, ax=axes[1], color='coral')
axes[1].set_title('Paid Games - Distribution of Avg Monthly Players')
axes[1].set_xlabel('Avg Monthly Players')
axes[1].set_ylabel('Number of Games')

plt.tight_layout()
plt.show()
```



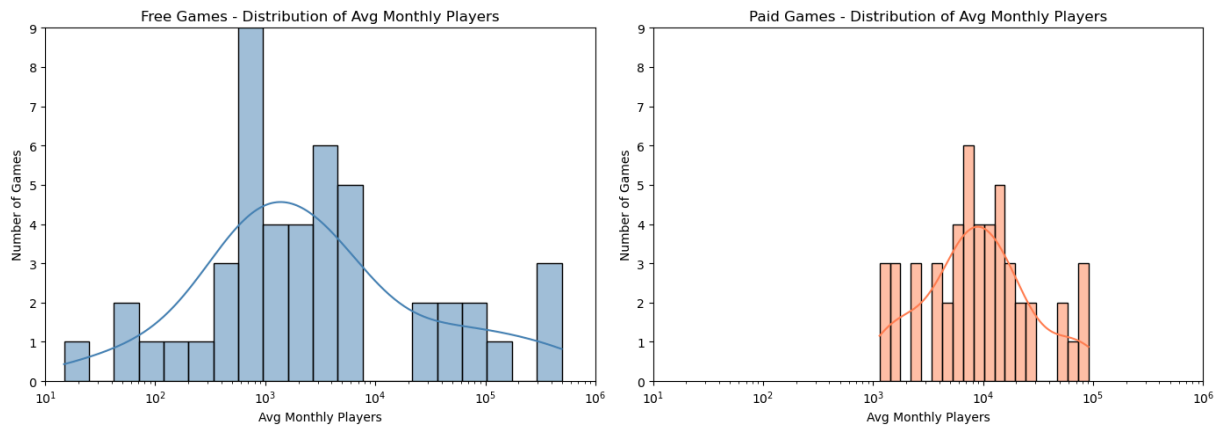
Both have extreme right skew, so to adjust for that, we'll plot it on the log scale.

```
In [9]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.histplot(df_free_agg, bins=20, kde=True, ax=axes[0], log_scale=True, color='steelblue')
axes[0].set_title('Free Games - Distribution of Avg Monthly Players')
axes[0].set_xlabel('Avg Monthly Players')
axes[0].set_ylabel('Number of Games')
axes[0].set_xlim(1e1, 1e6)
axes[0].set_ylim(0, 9)

sns.histplot(df_paid_agg, bins=20, kde=True, ax=axes[1], log_scale=True, color='coral')
axes[1].set_title('Paid Games - Distribution of Avg Monthly Players')
axes[1].set_xlabel('Avg Monthly Players')
axes[1].set_ylabel('Number of Games')
axes[1].set_xlim(1e1, 1e6)
axes[1].set_ylim(0, 9)

plt.tight_layout()
plt.show()
```



While the distributions are similar, it's apparent the paid games are more tightly bunched with games not having a big spread of average players in comparison to free games.

Let's look at game activity over time. To account for outliers in our dataset, we'll look at *median* average concurrent players.

```
In [10]: df_free_players['year'] = pd.to_datetime(df_free_players['year_month']).dt.year
df_paid_players['year'] = pd.to_datetime(df_paid_players['year_month']).dt.year

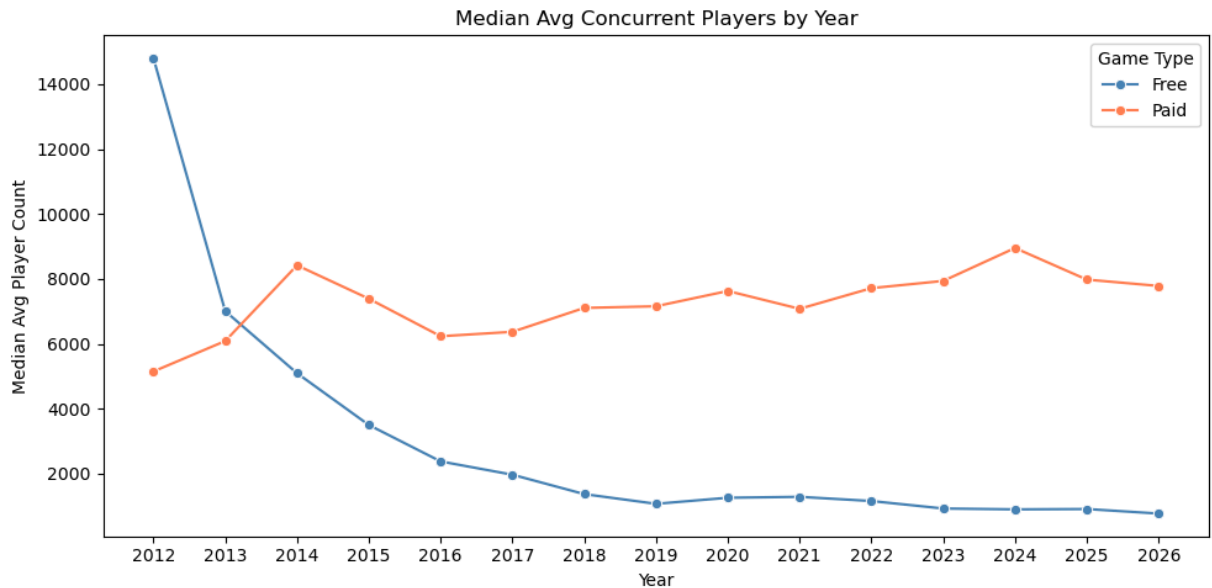
free_yearly = df_free_players.groupby('year')['average_player_count'].median()
paid_yearly = df_paid_players.groupby('year')['average_player_count'].median()

fig, ax = plt.subplots(figsize=(10, 5))

sns.lineplot(x=free_yearly.index, y=free_yearly.values, ax=ax, color='steelblue', marker='o')
sns.lineplot(x=paid_yearly.index, y=paid_yearly.values, ax=ax, color='coral', marker='o')

ax.set_title('Median Avg Concurrent Players by Year')
ax.set_xlabel('Year')
ax.set_ylabel('Median Avg Player Count')
ax.set_xticks(sorted(set(free_yearly.index) | set(paid_yearly.index)))
ax.legend(title='Game Type')

plt.tight_layout()
plt.show()
```

Overtime, we see the median for free games drops drastically over time while paid games maintain a relatively stable trend. This perhaps suggests free games were much more popular back in the early 2010s but lost steam over time.

3.3 Retention ratios

Now, let's look at the spread of median player counts.

```
In [11]: free_yearly = df_free_players.groupby('game_name')['average_player_count'].median()
paid_yearly = df_paid_players.groupby('game_name')['average_player_count'].median()

free_yearly['type'] = 'Free'
paid_yearly['type'] = 'Paid'

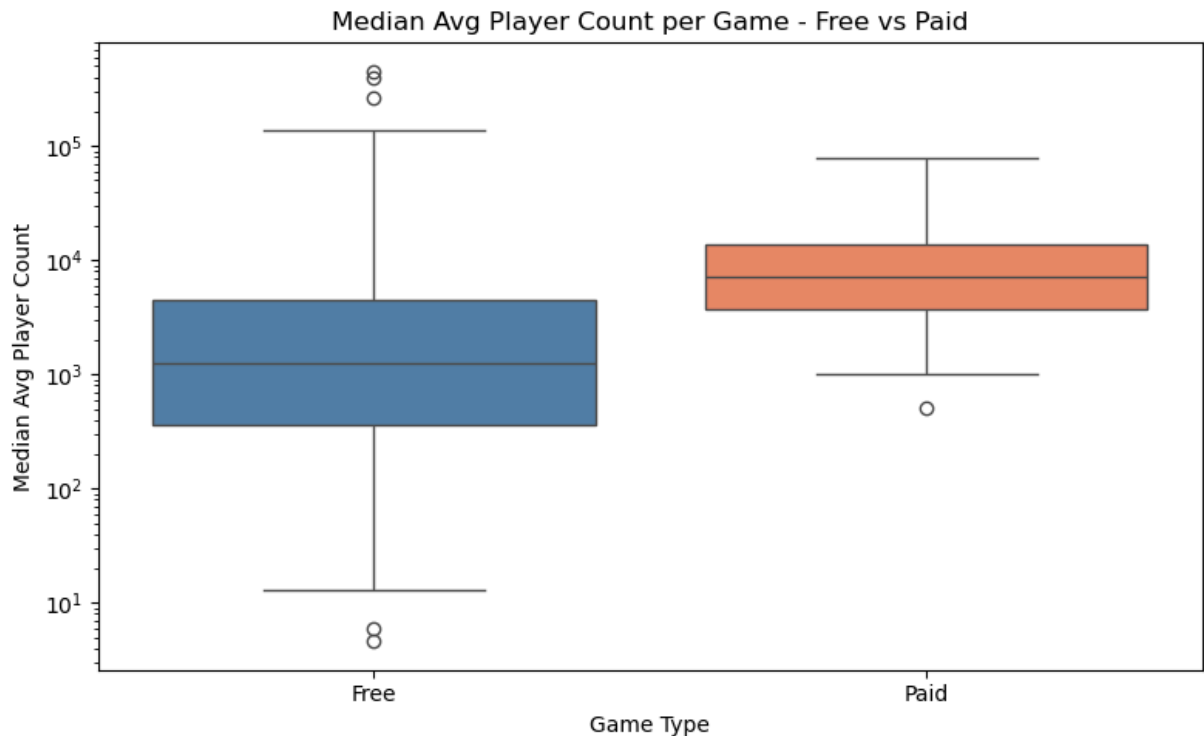
combined = pd.concat([free_yearly, paid_yearly])

fig, ax = plt.subplots(figsize=(8, 5))

sns.boxplot(data=combined, x='type',
            y='average_player_count',
            log_scale=True,
            hue='type',
            legend=False,
            palette={'Free': 'steelblue', 'Paid': 'coral'},
            ax=ax)

ax.set_title('Median Avg Player Count per Game - Free vs Paid')
ax.set_xlabel('Game Type')
ax.set_ylabel('Median Avg Player Count')

plt.tight_layout()
plt.show()
```



In comparison, paid games have smaller spread while free games have much larger spread. Due to the free pricing model, this makes sense. There may be well known free-to-play games that dominate the free-to-play playerbase.

Since we're interested in player retention, we need to transform our data to look at that then we can plot the distribution of retention ratios. Note that we are interested in the average player counts rather than peak when considering the retention ratio.

```
In [12]: retention_free = eda.compute_retention(df_free_players)
         retention_paid = eda.compute_retention(df_paid_players)

         retention_paid = retention_paid.replace( [np.inf, -np.inf], np.nan)
         retention_paid = retention_paid.dropna()
         retention_free = retention_free.replace( [np.inf, -np.inf], np.nan)
         retention_free = retention_free.dropna()

         print(retention_free.head(), '\n\n', retention_paid.head())
```

	first_players	last_players	retention_ratio
game_name			
APB Reloaded	2482.45	225.48	0.090830
Action Taimanin	834.75	781.38	0.936065
AdVenture Capitalist	11629.53	1046.73	0.090006
Apex Legends	74957.26	124387.18	1.659441
BATTALION: Legacy	0.82	2.13	2.597561

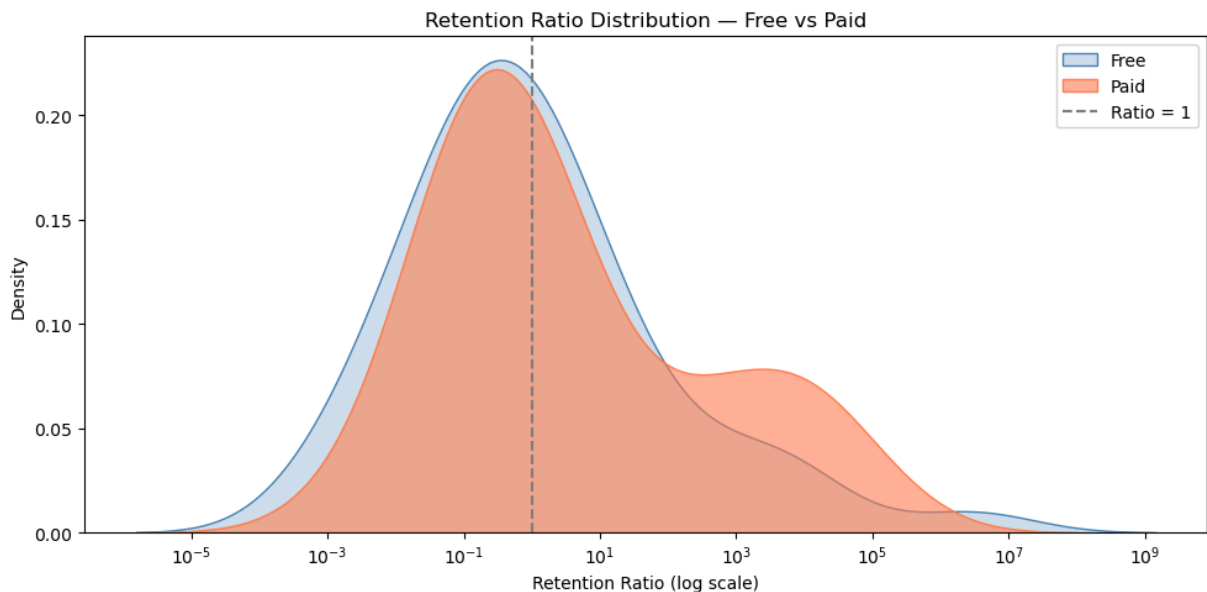
	first_players	last_players	retention_ratio
game_name			
Assetto Corsa	1.74	9828.48	5648.551724
Battlefield V	1163.34	3868.19	3.325073
BeamNG.drive	88.27	19526.97	221.218647
Black Myth: Wukong	970442.70	7854.38	0.008094
Borderlands 2	61242.78	2601.24	0.042474

```
In [13]: fig, ax = plt.subplots(figsize=(10, 5))

sns.kdeplot(retention_free['retention_ratio'], ax=ax, color='steelblue', fill=True,
sns.kdeplot(retention_paid['retention_ratio'], ax=ax, color='coral', fill=True, lab

ax.axvline(x=1, color='gray', linestyle='--', label='Ratio = 1')
ax.set_title('Retention Ratio Distribution — Free vs Paid')
ax.set_xlabel('Retention Ratio (log scale)')
ax.set_ylabel('Density')
ax.legend()

plt.tight_layout()
plt.show()
```



An important thing to note is that a ratio of 1 means that the player retention has stayed the same since release. Generally speaking, both free and games have almost a similar distribution with some key differences. Both have their peaks between 10^{-1} and 10^0 , suggesting more games see decline over time. A significant difference is the fatter tail paid games have, suggesting that there exists a sizable number of games that actually see growth rather than decline.

Due to the existence of outliers in our dataset, we may be interested in filtering out games that must have a minimum of 100 players average on release.

```
In [14]: retention_free_filtered = retention_free[retention_free['first_players'] > 100]
         retention_paid_filtered = retention_paid[retention_paid['first_players'] > 100]

         print(f"Free games: {len(retention_free)} -> {len(retention_free_filtered)} after f
         print(f"Paid games: {len(retention_paid)} -> {len(retention_paid_filtered)} after f

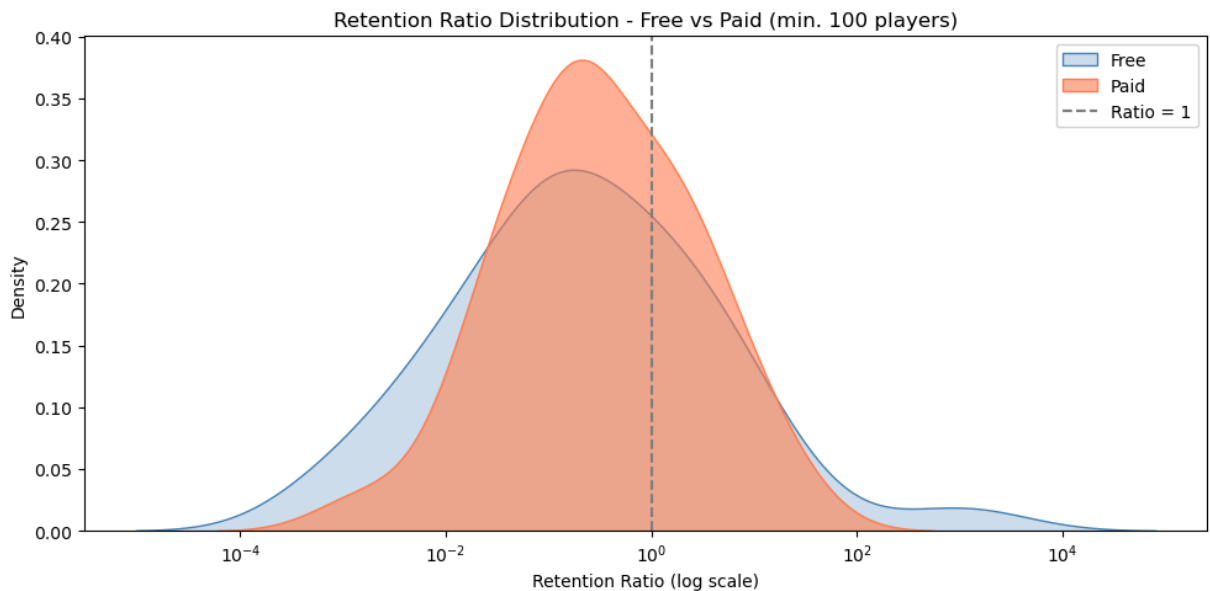
Free games: 46 -> 35 after filter
Paid games: 49 -> 36 after filter
```

```
In [15]: fig, ax = plt.subplots(figsize=(10, 5))

         sns.kdeplot(retention_free_filtered['retention_ratio'], ax=ax, color='steelblue', f
         sns.kdeplot(retention_paid_filtered['retention_ratio'], ax=ax, color='coral', fill=

         ax.axvline(x=1, color='gray', linestyle='--', label='Ratio = 1')
         ax.set_title('Retention Ratio Distribution - Free vs Paid (min. 100 players)')
         ax.set_xlabel('Retention Ratio (log scale)')
         ax.set_ylabel('Density')
         ax.legend()

         plt.tight_layout()
         plt.show()
```



Paid games have a much tighter distribution while free games have a wider one. Free games have extremes with games seeing major loss but also major growth. Despite our analysis not going into the details of genres, like being live service, we assume this may be a large contributing factor.

4 Statistical Inference

For the following, we choose significance level $\alpha = 0.05$ for our analysis.

4.1 Means of retention ratios

Something we're interested in is asking is whether the mean retention is truly different between free-to-play and paid games. As such, we'll use Welch's t-test as we do not want to assume equal variances between the two groups.

Our set-up is as follows:

- $H_0 : \mu_{\text{free}} = \mu_{\text{paid}}$
- $H_a : \mu_{\text{free}} \neq \mu_{\text{paid}}$ where "free" and "paid" represent the free-to-play and paid games, respectively.

```
In [16]: # Welch's t-test on raw retention ratio
t_stat, p_value = stats.ttest_ind(
    retention_free['retention_ratio'],
    retention_paid['retention_ratio'],
    equal_var=False
)

print("t-statistic =", t_stat)
print("p-value =", p_value)

# Welch's t-test on log retention ratio
t_stat_log, p_value_log = stats.ttest_ind(
    np.log(retention_free['retention_ratio']),
    np.log(retention_paid['retention_ratio']),
    equal_var=False
)

print("\nlog t-statistic =", t_stat_log)
print("log p-value =", p_value_log)
```

```
t-statistic = 0.9231781462117606
p-value = 0.36081029761229655
```

```
log t-statistic = -1.281385210712014
log p-value = 0.20324476278890644
```

For both tests on the raw and log-scaled data, both p-values are greater than our chosen $\alpha = 0.05$, thus we fail to reject our null hypothesis. In other words, the means between the two groups are the same. This lines up with our inspection of the distribution visualizations.

4.2 True difference between retention ratios

Since Welch's t-test did not give us meaningful insight, another question we may be interested in is whether two randomly chosen games from free-to-play and paid consistently have higher/lower retention ratios than the other. In other words, do they come from the same distribution? Due to the normality assumption, we'll work with the log-scaled data.

Our set-up is as follows:

- H_0 : The two distributions are identical
- H_a : The two distributions are not identical

```
In [17]: retention_free_log = np.log(retention_free['retention_ratio'])
         retention_paid_log = np.log(retention_paid['retention_ratio'])

         u_stat, p_value_mw = stats.mannwhitneyu(retention_free_log, retention_paid_log, alt='not equal')
         print("U-statistic =", u_stat)
         print("p-value =", p_value_mw)
```

U-statistic = 980.0

p-value = 0.2752829142706459

Given the test's p-value ≈ 0.275 , this is greater than our $\alpha = 0.05$, so we fail to reject the null hypothesis. In other words, the distributions are identical. However, from our analysis of the distributions in the EDA section, we noticed different shapes, so this may be due to the lack of significant evidence. Regardless, we cannot conclude that paid or free-to-play games exhibit stronger long-term player retention than the other.

4.3 Confidence intervals for mean log retention

Now, based on the log retention ratios, let's construct a confidence interval for the difference in means where our parameter of interest is

$$\mu_{\text{free}} - \mu_{\text{paid}}$$

```
In [18]: n1 = len(retention_free_log)
         n2 = len(retention_paid_log)

         mean1 = retention_free_log.mean()
         mean2 = retention_paid_log.mean()

         var1 = retention_free_log.var(ddof=1)
         var2 = retention_paid_log.var(ddof=1)

         print("Mean log retention (free):", mean1)
         print("Mean log retention (paid):", mean2)
```

```

# Standard error
SE = np.sqrt(var1/n1 + var2/n2)

# Welch-Satterthwaite degrees of freedom
df_welch = (
    (var1/n1 + var2/n2)**2
) / (
    ((var1/n1)**2)/(n1-1)
    +
    ((var2/n2)**2)/(n2-1)
)

# Critical value
t_star = stats.t.ppf(0.975, df_welch)

# 95% CI for difference in means
difference = mean1 - mean2
lower = difference - t_star * SE
upper = difference + t_star * SE

print("\nDifference (free - paid) =", difference)
print("95% CI = [", lower, ",", upper, "]")

```

Mean log retention (free): 0.08021965720029552

Mean log retention (paid): 1.2771924113835933

Difference (free - paid) = -1.1969727541832977

95% CI = [-3.0519625355398112 , 0.6580170271732155]

In this case, the confidence interval contains 0. This is consistent with our results from the Welch and Mann-Whitney tests earlier that there is no significant difference between the two groups: free-to-play and paid. With 95% confidence, the true difference in their retentions must lie between $[-3.05, 0.66]$. An interesting observation is the wide interval, suggesting some variance in the data. Additionally, since the interval includes negative and positive ends, it also suggests that even free-to-play games have better retention than paid.

4.4 Bootstrapping

The parametric 95% confidence interval in 4.3 relies on log-retention being approximately normal. Since our sample sizes are modest ($n_{\text{free}} = 46$, $n_{\text{paid}} = 49$) and the distributions are visibly heavy-tailed, we re-estimate the interval with a nonparametric percentile bootstrap to check whether the parametric result depends on those assumptions.

The procedure is to:

1. Resample each group's per-game log-retention values with replacement
2. Recompute the statistic of interest
3. Repeat $B = 10,000$ times
4. Read the central 95% of the bootstrap distribution

Because our analysis unit is one number per game, no clustering correction is needed beyond resampling games directly. We bootstrap both the mean difference (to compare to 4.3) and the median difference as a more robust compliment.

```
In [19]: B = 10_000
rng = np.random.default_rng(189)

log_free_np = np.log(retention_free['retention_ratio'].to_numpy())
log_paid_np = np.log(retention_paid['retention_ratio'].to_numpy())

rows = []
for name, stat in [('mean', np.mean), ('median', np.median)]:
    diff_boot = stat_inf.boot_diff(log_free_np, log_paid_np, stat, B, rng)
    pt = stat(log_free_np) - stat(log_paid_np)
    lo, hi = stat_inf.pct_ci(diff_boot)
    rows.append({'statistic': f'{name} log(retention)',
                'point_estimate (Free - Paid)': round(pt, 3),
                '95% CI lower': round(lo, 3),
                '95% CI upper': round(hi, 3),
                'ratio scale': f"{np.exp(pt):.2f}x [{np.exp(lo):.2f}, {np.exp(hi)

pd.DataFrame(rows)
```

```
Out[19]:
```

	statistic	point_estimate (Free - Paid)	95% CI lower	95% CI upper	ratio scale
0	mean log(retention)	-1.197	-3.011	0.573	0.30x [0.05, 1.77]
1	median log(retention)	-0.579	-2.433	1.422	0.56x [0.09, 4.14]

The bootstrap mean-difference CI on the log scale is $[-3.01, 0.57]$, which is almost identical to the parametric Welch confidence interval $[-3.05, +0.66]$ from 4.3, so the t-based interval was reliable for this data.

We ran the test on $\log(\text{retention ratio})$, so the null hypothesis "the two groups have equal mean retention" corresponds to a mean log-difference of 0. Back-transforming via $\exp(\cdot)$, the equivalent statement on the original retention-ratio scale is that the ratio of typical-free to typical-paid retention equals 1. So a CI containing 0 on the log scale is the same statement as a CI containing 1 on the ratio scale.

The log-scale point estimate of -1.197 back-transforms to $\exp(-1.197) \approx 0.30$, which is the **0.30x** entry in the **ratio scale** column - the bootstrap's best single guess is that the typical free game's retention is about 30% of the typical paid game's. The 95% CI is very wide on the ratio scale, running from 0.05 (free much worse) to 1.77 (free moderately better), and the null value 1 sits inside it. The median-difference row tells the same story with more uncertainty: log-CI $[-2.43, +1.42]$, ratio-CI $[0.09, 4.14]$. Since both CIs include the null, at $\alpha = 0.05$ we fail to reject equal retention, consistent with the previous analyses.

4.4.1 Retention by post-peak window

The single-number summary above averages across each game's entire post-launch history, which can hide time-scale effects: a launch-spike pattern (early retention high, late retention low) and a slow-burn pattern can produce the same overall ratio while behaving completely differently. To check this, we split each game's life into five post-peak windows and compute a window-specific retention ratio as

$$r_w = \frac{\text{mean average players in window } w}{\text{game's peak monthly average}}$$

A game contributes to a window only if its observed history extends into it. We then bootstrap the median free – paid difference inside each window, with $B = 10,000$ and the same percentile method.

```
In [20]: # windowed retention (one row per game per group)
WINDOWS = [(1, 6), (7, 12), (13, 24), (25, 48), (49, 96)]

w_free = stat_inf.windowed_retention(df_free_players, 'Free')
w_paid = stat_inf.windowed_retention(df_paid_players, 'Paid')

rows = []
for lo, hi in WINDOWS:
    col = f'{lo}-{hi}'
    fr = w_free[col].dropna().to_numpy()
    pa = w_paid[col].dropna().to_numpy()
    diff_boot = stat_inf.boot_diff(fr, pa, np.median, B, rng)
    pt = float(np.median(fr) - np.median(pa))
    lo_ci, hi_ci = stat_inf.pct_ci(diff_boot)
    rows.append({'window (mo post-peak)': col,
                'n_free': len(fr), 'n_paid': len(pa),
                'median Free': round(float(np.median(fr)), 3),
                'median Paid': round(float(np.median(pa)), 3),
                'Free - Paid': round(pt, 3),
                '95% CI': f"[{lo_ci:+.3f}, {hi_ci:+.3f}]",
                'excludes 0?': 'yes' if (lo_ci > 0 or hi_ci < 0) else 'no'})

windowed_summary = pd.DataFrame(rows)
windowed_summary
```

Out[20]:

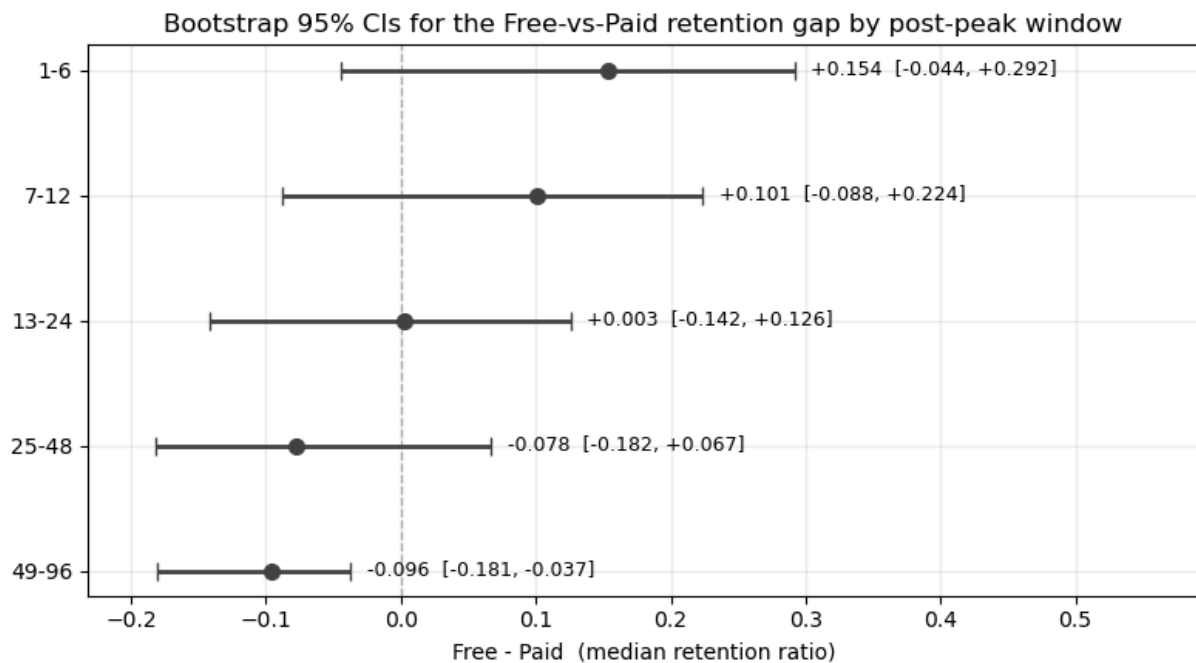
	window (mo post-peak)	n_free	n_paid	median Free	median Paid	Free - Paid	95% CI	excludes 0?
0	1-6	47	50	0.569	0.414	0.154	[-0.044, +0.292]	no
1	7-12	44	47	0.387	0.287	0.101	[-0.088, +0.224]	no
2	13-24	42	45	0.279	0.275	0.003	[-0.142, +0.126]	no
3	25-48	40	35	0.180	0.258	-0.078	[-0.182, +0.067]	no
4	49-96	30	28	0.115	0.211	-0.096	[-0.181, -0.037]	yes

Interestingly enough, it appears for months 1-12, free games have better retention than paid games. However, they equalize around the 13-24 month mark, and after that, paid games increase in retention over free-to-play games after 25 months (long-term). This suggests free-to-play games spike early but decay faster while paid games decay at a slower rate.

Another important thing to note is the shrinking size of the game pool over time. Due to the size of the data, less games will actually have long-term data available, which may not necessarily be reflective of the Steam games population as these are the top fifty games.

```
In [21]: # Forest plot of the Free - Paid difference by window
labels = [r['window (mo post-peak)'] for r in rows]
pts = [r['Free - Paid'] for r in rows]
los = [float(r['95% CI'].split(',')[0].strip(' ')) for r in rows]
his = [float(r['95% CI'].split(',')[1].strip(' ')) for r in rows]

fig, ax = plt.subplots(figsize=(8, 4.5))
y = np.arange(len(labels))
ax.axvline(0, ls='--', color='#aaa', lw=1)
ax.errorbar(pts, y,
            xerr=[np.array(pts) - np.array(los), np.array(his) - np.array(pts)],
            fmt='o', color='#444', ecolor='#444', capsize=4, lw=2, markersize=7)
for i, (pt, lo, hi) in enumerate(zip(pts, los, his)):
    ax.text(hi + 0.012, i, f"{pt:+.3f} [{lo:+.3f}, {hi:+.3f}]",
            va='center', fontsize=9)
ax.set_yticks(y); ax.set_yticklabels(labels); ax.invert_yaxis()
ax.set_xlabel('Free - Paid (median retention ratio)')
ax.set_title('Bootstrap 95% CIs for the Free-vs-Paid retention gap by post-peak win')
ax.grid(alpha=.25); ax.set_axisbelow(True)
ax.set_xlim(min(los) - 0.05, max(his) + 0.30)
plt.tight_layout(); plt.show()
```



Visually, this further supports the analysis above. Free-to-play games sit above paid games in the first year (free – paid ≈ 0.15 at 1-6 months, $+0.10$ at 7-12 months), the gap closes near month 13-24, and then the sign begins to flip by 25-28 months. By 49-96 months post-peak the median paid game retains roughly 9.6 percentage points more of its peak than the median free game, with a 95% bootstrap CI of $[-0.181, -0.037]$ that excludes 0.

So at $\alpha = 0.05$ we reject equal retention at long horizons, even though the comparisons in 4.1-4.4 fail to reject. However, the two findings are not contradictory: the aggregate averages out a positive short-term gap and a negative long-term gap, which roughly cancel.

There must be a separation. Free-to-play games hold a larger share of their peak in the launch year but lose it faster after, and the gap becomes statistically detectable only once we let time enter the analysis.

4.5 Review scores

The outcome is the within-game review-score change, defined as `latest_score - release_score`.

Each game contributes two scores: its release-window score and its latest all-time Steam review score. Because the two scores come from the same game, the inference uses a paired t-test. The free-to-play and paid lists are analyzed separately, with 50 games in each group.

```
In [22]: review_scores = stat_inf.load_review_scores()
paired_scores = review_scores.dropna(subset=["release_score", "latest_score"]).copy

analysis_columns = [
    "game_name",
```

```

    "group",
    "release_score",
    "latest_score",
    "score_change",
    "Shifted",
    "Capped",
]

print(f"Games analyzed: {paired_scores.shape[0]}")
print(f"Free-to-play games: {(paired_scores['group'] == 'Free-to-play').sum()}")
print(f"Paid games: {(paired_scores['group'] == 'Paid').sum()}")
print()
print(paired_scores[analysis_columns].head(10).to_string(index=False))

summary = paired_scores.groupby("group")[["release_score", "latest_score", "score_c
summary

```

Games analyzed: 100
 Free-to-play games: 50
 Paid games: 50

	game_name	group	release_score	latest_score	score_change	Shifted
Capped						
True	Counter-Strike 2	Free-to-play	94	86	-8	True
True	Dota 2	Free-to-play	92	81	-11	True
True	PUBG: BATTLEGROUNDS	Free-to-play	44	60	16	False
True	Apex Legends	Free-to-play	86	68	-18	False
False	Warframe	Free-to-play	94	88	-6	True
False	War Thunder	Free-to-play	92	72	-20	True
True	Lost Ark	Free-to-play	75	70	-5	False
True	Unturned	Free-to-play	84	91	7	False
True	Team Fortress 2	Free-to-play	96	91	-5	True
False	Overwatch 2	Free-to-play	80	80	0	False

Out[22]:

	release_score				latest_score					
	count	mean	median	std	count	mean	median	std	count	mean
group										
Free-to-play	50	79.58	84.0	15.828275	50	79.56	81.0	12.972750	50	-0.02
Paid	50	88.50	96.0	13.669943	50	91.20	94.5	7.835034	50	2.70

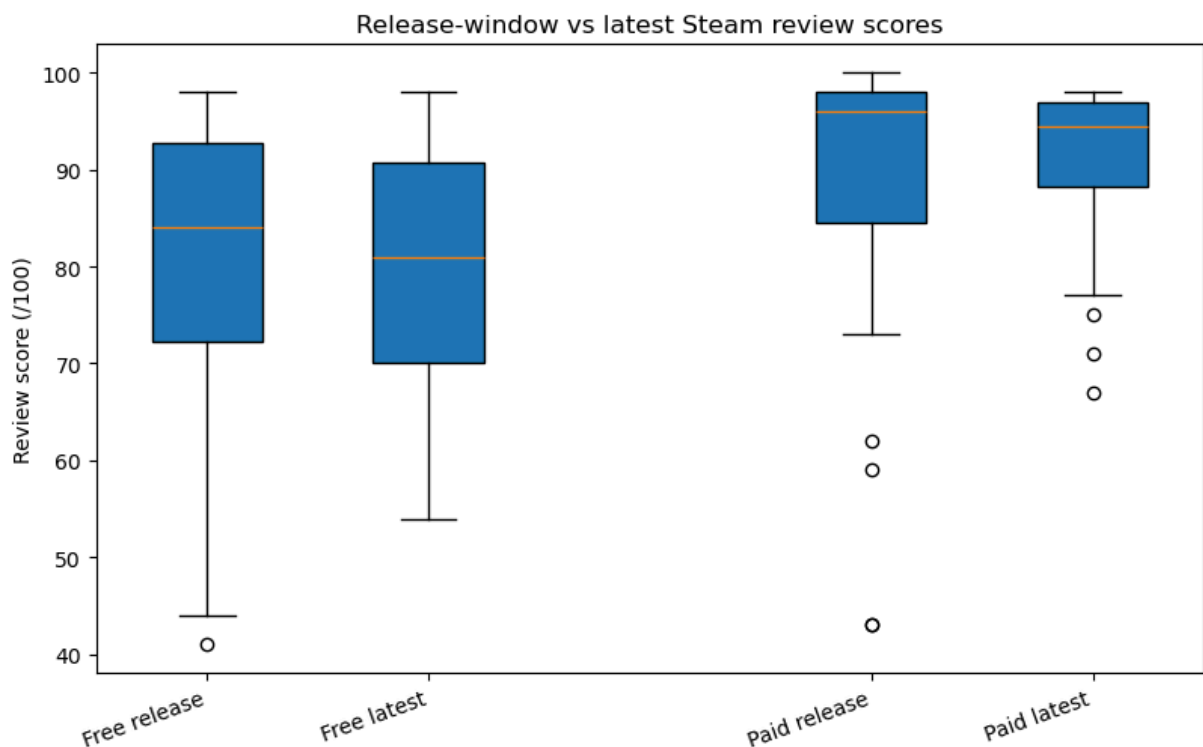


There are some important notes about the data processed.

- The paired t-test is release vs latest within the same game, not paid vs free against each other.
- The free-to-play group has 50 paired games and the paid group has 50 paired games.
- `Shifted = True` means the game was released before Steam reviews existed, so the release-window score starts when Steam reviews launched rather than on the true release date.
- `Capped = True` means the release-window score is based on a capped sample of review pages.

```
In [23]: plot_positions = [1, 2, 4, 5]
plot_data = [
    paired_scores.loc[paired_scores["group"] == "Free-to-play", "release_score"],
    paired_scores.loc[paired_scores["group"] == "Free-to-play", "latest_score"],
    paired_scores.loc[paired_scores["group"] == "Paid", "release_score"],
    paired_scores.loc[paired_scores["group"] == "Paid", "latest_score"],
]

fig, ax = plt.subplots(figsize=(8.2, 5.2))
ax.boxplot(plot_data, positions=plot_positions, patch_artist=True)
ax.set_xticks(plot_positions)
ax.set_xticklabels(["Free release", "Free latest", "Paid release", "Paid latest"],
ax.set_ylabel("Review score (/100)")
ax.set_title("Release-window vs latest Steam review scores")
fig.tight_layout()
```

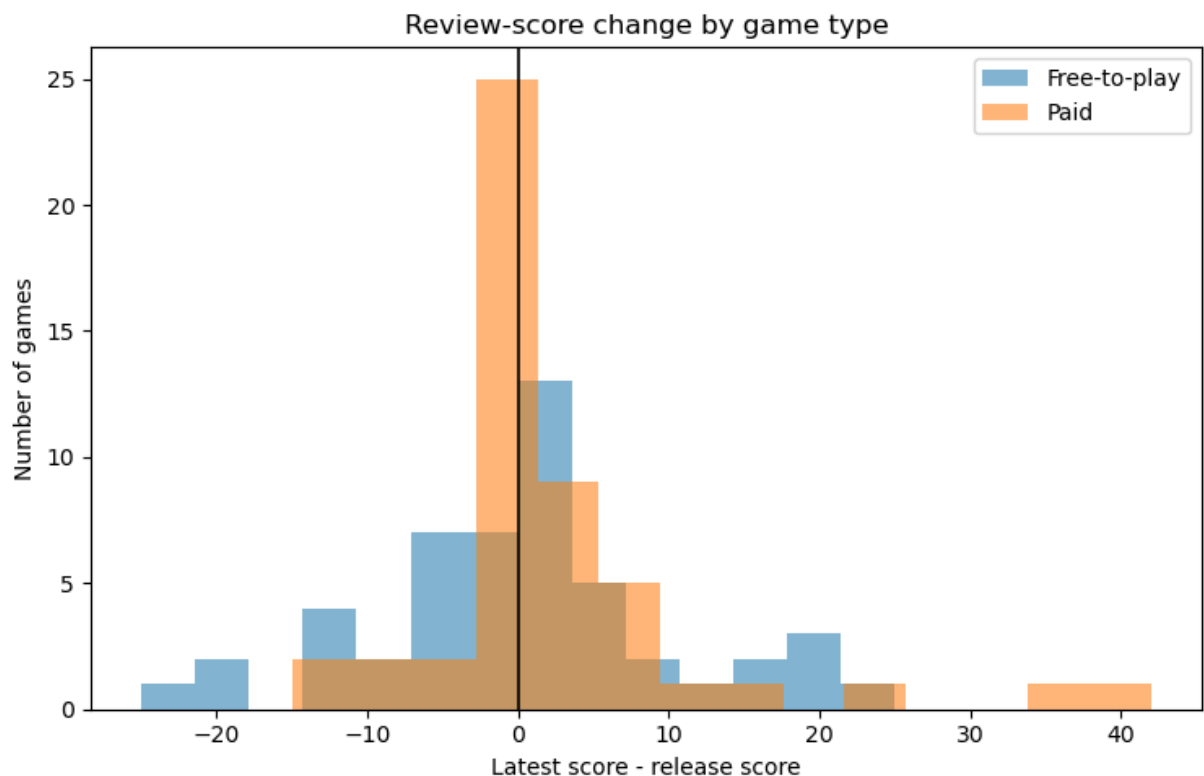


The boxplots compare release-window and latest review scores within each game type.

```
In [24]: plot_groups = ["Free-to-play", "Paid"]

fig, ax = plt.subplots(figsize=(7.6, 5.0))
for group in plot_groups:
    group_changes = paired_scores.loc[paired_scores["group"] == group, "score_change"]
    ax.hist(group_changes, bins=14, alpha=0.55, label=group)

ax.axvline(0, color="black", linewidth=1.2)
ax.set_xlabel("Latest score - release score")
ax.set_ylabel("Number of games")
ax.set_title("Review-score change by game type")
ax.legend()
fig.tight_layout()
```



The t-tests use each game's score change. A positive value means the latest review score is higher than the release-window score.

Now, to conduct a paired t-test by game type:

- H_0 : within a game type, the average release-to-latest score change is 0
- H_a : within a game type, the average release-to-latest score change is not 0

```
In [25]: test_results = pd.DataFrame({
    group: stat_inf.run_paired_test(group_frame)
    for group, group_frame in paired_scores.groupby("group")
}).T
test_results.index.name = "group"

result_columns = [
    "games",
```

```

    "release_mean",
    "latest_mean",
    "mean_change",
    "t_statistic",
    "p_value",
    "ci_low",
    "ci_high",
    "cohens_d",
    "shapiro_p",
    "wilcoxon_p",
    "shifted_games",
    "capped_games",
]

test_results = test_results[result_columns]
test_results.round(4)

```

Out[25]:

	games	release_mean	latest_mean	mean_change	t_statistic	p_value	ci_low	ci_high
--	-------	--------------	-------------	-------------	-------------	---------	--------	---------

group

Free-to-play	50.0	79.58	79.56	-0.02	-0.0141	0.9888	-2.8634	2.82
Paid	50.0	88.50	91.20	2.70	1.9777	0.0536	-0.0436	5.44



In [26]:

```

plot_groups = ["Free-to-play", "Paid"]
change_frame = paired_scores.groupby("group")["score_change"].agg(["mean", "std", "count"])
change_frame["se"] = change_frame["std"] / np.sqrt(change_frame["count"])
change_frame["critical_value"] = stats.t.ppf(0.975, change_frame["count"] - 1)
change_frame["ci"] = change_frame["critical_value"] * change_frame["se"]

fig, ax = plt.subplots(figsize=(7.2, 4.8))
ax.bar(change_frame.index, change_frame["mean"], yerr=change_frame["ci"], capsize=8)
ax.axhline(0, color="black", linewidth=1.2)
ax.set_ylabel("Mean latest - release score")
ax.set_title("Mean review-score change with 95% CI")
fig.tight_layout()

change_frame[["mean", "std", "count", "ci"]]

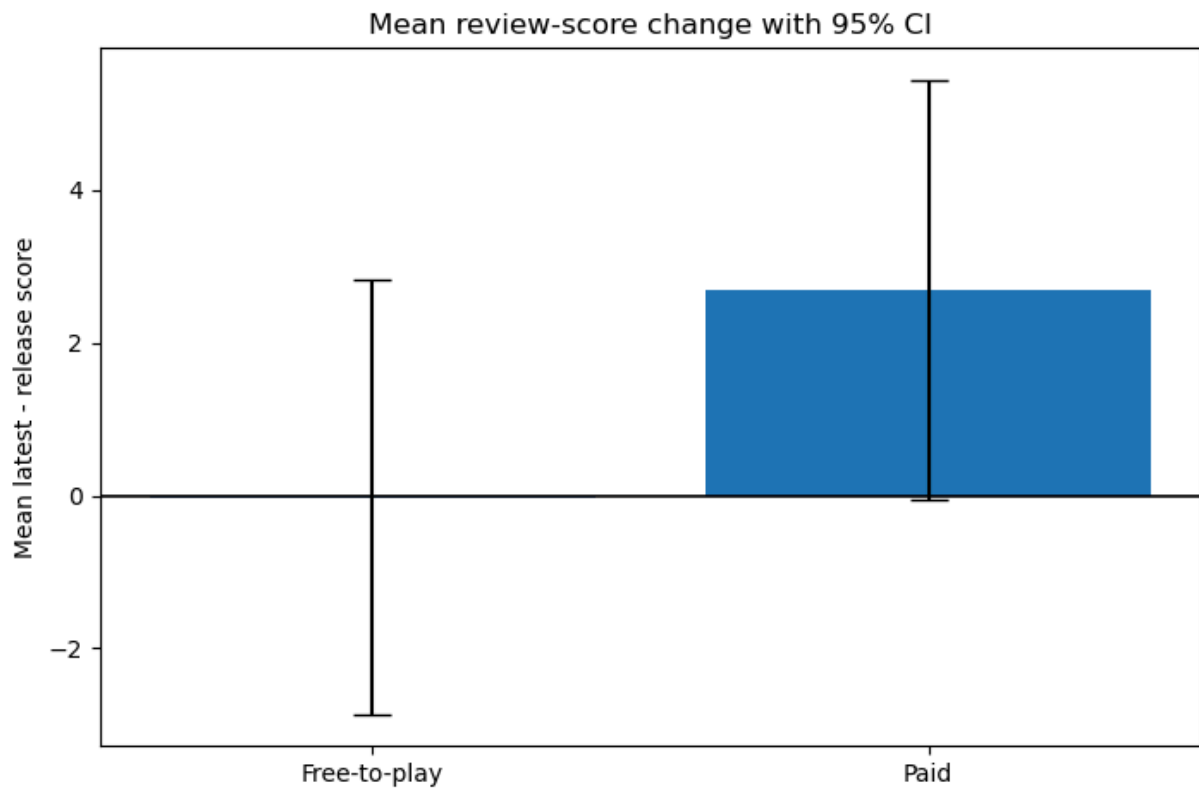
```

Out[26]:

	mean	std	count	ci
--	------	-----	-------	----

group

Free-to-play	-0.02	10.005080	50	2.843412
Paid	2.70	9.653698	50	2.743551



Free-to-play mean change = -0.02 points ($p = 0.989$); no detectable difference between release-window and latest review scores. Paid mean change = 2.70 points ($p = 0.054$); a small increase after release that is not significant at the 0.05 level (Cohen's $d \approx 0.28$). `mean_change` is calculated as `latest_score - release_score`, so positive values mean review scores increased after release.

The Shapiro-Wilk and Wilcoxon p-values are included because the score-change distributions may not be normal; the paid group's low Shapiro-Wilk p-value flags non-normality, so its Wilcoxon p-value (0.087) is the more reliable check and agrees with the t-test. The shifted pre-2013 windows should be treated as a caveat because those release-window scores are not true launch-window scores.

4.6 Modeling retention as a function of multiple variables

First, we'll be rebuilding retention ratios for regression analysis.

```
In [27]: players, games = stat_inf.load_tables()
features = stat_inf.build_game_features(players, games)

analysis_columns = [
    "game_name",
    "group",
    "release_year",
    "estimated_owners_mid",
    "launch_peak_concurrent_players",
```



```

        "peak_avg",
        "recent_avg",
        "retention_ratio",
    ]

    summary = stat_inf.summarize_by_group(
        features,
        ["retention_ratio", "peak_avg", "recent_avg", "release_year", "estimated_owners"]
    )

```

There are some important notes about the data processed.

- The dataset provided has an estimated owners column, so to make it friendly to analyze, this was modified to be a column that takes the middle of the specified range of ownership.
- We originally wanted to include whether a game was considered "multiplayer" and review scores, but this data was not obtained. Therefore, we have less variables to consider.
- `estimated_owners_mid` is the midpoint of the estimated owner range from the Steam list data. For example, 5M - 10M becomes 7.5M, and 20M - 50M becomes 35M. It gives the range a single numeric value so we can use it in correlation and regression analyses.

Now, fit a multivariable regression using the predictors available in `data/`.

```

In [28]: regression_frame = features.dropna(subset=[
        "log_retention_ratio",
        "is_free_to_play",
        "release_year_centered",
        "log_estimated_owners",
        "log_launch_peak_concurrent_players",
    ]).copy()

formula = "log_retention_ratio ~ is_free_to_play + release_year_centered + log_esti
sm_model = smf.ols(formula, data=regression_frame).fit()
print(sm_model.summary())

```

OLS Regression Results

```

=====
Dep. Variable:    log_retention_ratio    R-squared:                0.250
Model:            OLS                    Adj. R-squared:            0.217
Method:            Least Squares         F-statistic:               7.580
Date:              Fri, 05 Jun 2026      Prob (F-statistic):       2.57e-05
Time:              16:14:42              Log-Likelihood:           -165.09
No. Observations: 96                    AIC:                       340.2
Df Residuals:      91                    BIC:                       353.0
Df Model:          4
Covariance Type:  nonrobust
=====

```

```

=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
Intercept                -1.3927      2.573      -0.541      0.590
-6.503      3.717
is_free_to_play           -0.4964      0.292     -1.702      0.092
-1.076      0.083
release_year_centered     -0.0981      0.040     -2.452      0.016
-0.178     -0.019
log_estimated_owners      -0.3481      0.393     -0.885      0.378
-1.129      0.433
log_launch_peak_concurrent_players  0.5707      0.130      4.397      0.000
0.313      0.829
=====

```

```

=====
Omnibus:                8.824    Durbin-Watson:           1.876
Prob(Omnibus):           0.012    Jarque-Bera (JB):         8.536
Skew:                    -0.653    Prob(JB):                  0.0140
Kurtosis:                 3.654    Cond. No.                  148.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Based on this model, it explains about 25% of the variance in the log retention ratio. Most interestingly, `log_launch_peak_concurrent_players` has very high significance ($p = 0.000$). This could be a signal of such games at launch being high quality games that made people want to stay. Additionally, release year is also significant ($p = 0.016$), which makes sense intuitively. As new games release, the traffic is likely to shift from old to new games, especially if players are satisfied with finishing one playthrough.

Given this model, we might be concerned about multicollinearity. In other words, what variables have high correlation with one another?

```

In [29]: corr_frame = features[[
            "retention_ratio",
            "release_year",
            "estimated_owners_mid",
            "launch_peak_concurrent_players",
            "peak_avg",

```

```

    "recent_avg",
]].dropna().copy()

corr_frame["log_estimated_owners_mid"] = np.log10(corr_frame["estimated_owners_mid"]
corr_frame["log_launch_peak_concurrent_players"] = np.log10(
    corr_frame["launch_peak_concurrent_players"].where(corr_frame["launch_peak_conc
)
corr_frame["log_peak_avg"] = np.log10(corr_frame["peak_avg"].where(corr_frame["peak
corr_frame["log_recent_avg"] = np.log10(corr_frame["recent_avg"].where(corr_frame["
corr_frame["log_retention_ratio"] = np.log(corr_frame["retention_ratio"].where(corr

corr_cols = [
    "log_retention_ratio",
    "release_year",
    "log_estimated_owners_mid",
    "log_launch_peak_concurrent_players",
    "log_peak_avg",
    "log_recent_avg",
]

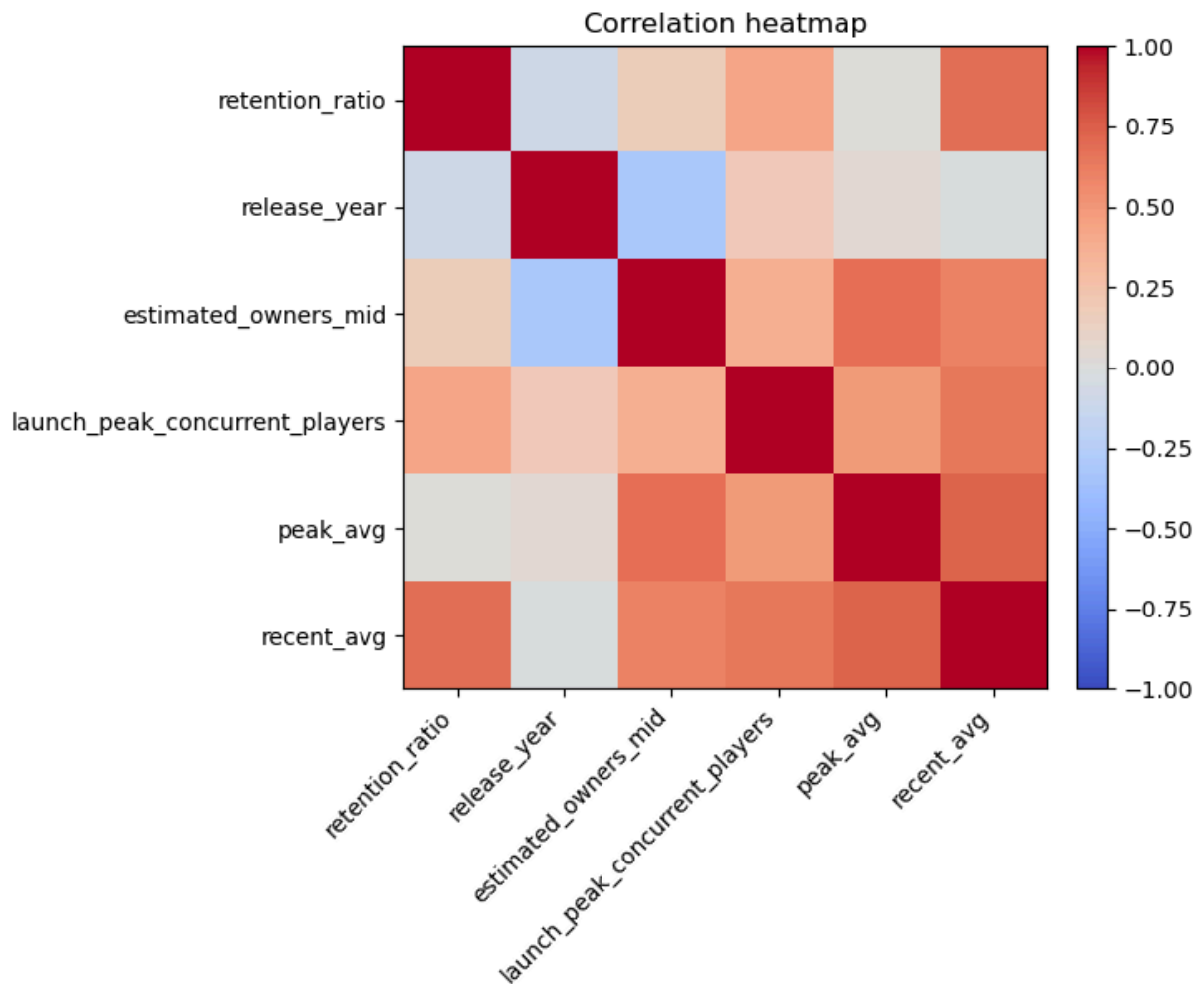
corr = corr_frame[corr_cols].replace([np.inf, -np.inf], np.nan).corr(numeric_only=T
strength = corr["log_retention_ratio"].drop("log_retention_ratio").abs().sort_value
print("Absolute Pearson correlation with log(retention ratio):")
print(strength.to_string(float_format=lambda x: f"{x:.3f}"))

fig, ax = plt.subplots(figsize=(7.2, 6.2))
im = ax.imshow(corr, vmin=-1, vmax=1, cmap="coolwarm")
ax.set_xticks(range(len(corr_cols)))
ax.set_yticks(range(len(corr_cols)))
ax.set_xticklabels([c.replace("log_", "") for c in corr_cols], rotation=45, ha="rig
ax.set_yticklabels([c.replace("log_", "") for c in corr_cols])
ax.set_title("Correlation heatmap")
fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
fig.tight_layout()

```

Absolute Pearson correlation with log(retention ratio):

log_recent_avg	0.681
log_launch_peak_concurrent_players	0.422
log_estimated_owners_mid	0.166
release_year	0.096
log_peak_avg	0.000



There are some relationships immediately noticeable:

- `launch_peak_concurrent_players` , `peak_avg` , and `recent_avg` have high correlation. This makes sense. A new and shiny game is likely to attract a whole array of players. Note that the latter two variables were not used in the regression analysis, so this is merely for observation.
- `estimated_owners_mid` seems to have moderate correlation with variables with player count. Owners are related to player counts, so this is another obvious relationship. Again, this was not included in the model, so this is not an issue.

4.7 Removing variables from model

```
In [30]: formula_reduced = (  
    "log_retention_ratio ~ "  
    "is_free_to_play + "  
    "release_year_centered + "  
    "log_launch_peak_concurrent_players"  
    )
```

```
reduced_model = smf.ols(formula_reduced, data=regression_frame).fit()
print(reduced_model.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:          log_retention_ratio    R-squared:                 0.243
Model:                  OLS                  Adj. R-squared:            0.219
Method:                 Least Squares         F-statistic:               9.868
Date:                  Fri, 05 Jun 2026       Prob (F-statistic):        1.05e-05
Time:                  16:14:44              Log-Likelihood:            -165.50
No. Observations:      96                   AIC:                      339.0
Df Residuals:          92                   BIC:                      349.3
Df Model:              3
Covariance Type:       nonrobust
=====
=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
Intercept                -3.6247      0.510      -7.107      0.000
-4.638    -2.612
is_free_to_play          -0.4925      0.291      -1.691      0.094
-1.071      0.086
release_year_centered    -0.0832      0.036      -2.295      0.024
-0.155    -0.011
log_launch_peak_concurrent_players  0.5197      0.116       4.474      0.000
0.289      0.750
=====
Omnibus:                 7.591    Durbin-Watson:           1.869
Prob(Omnibus):           0.022    Jarque-Bera (JB):         7.091
Skew:                   -0.610    Prob(JB):                 0.0288
Kurtosis:               3.534    Cond. No.                  16.1
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

`log_estimated_owners` was removed from the model because it had the highest p-value in the original regression and did not seem to contribute much to explaining retention.

After removing it, the model performed about the same as before ($R^2 = 0.243$). Release year and launch peak concurrent players were still the strongest predictors of retention. Free-to-play status showed some relationship with retention, but its effect was weaker than the other two variables.

Overall, this suggests that when a game was released and how popular it was at launch are more closely related to long-term retention than estimated ownership.

5 Results

Free-to-play and paid games tell different stories depending on how far out you look.

In the short run (the first year after a game peaks) free-to-play games actually hold onto players slightly better than paid games. But that advantage fades. By the 3-8 year mark, paid games retain a meaningfully larger share of their peak playerbase, and this is the only finding in the analysis that clears the bar for statistical significance.

Zooming out to the full dataset, neither pricing model is a clear winner overall. The distributions overlap too heavily and the sample is too small (50 games per group) to draw a firm conclusion at the aggregate level. What retention differences do exist get averaged out when you collapse everything into a single number.

The regression adds useful context: the biggest predictor of long-term retention isn't whether a game is free or paid. What matters more is how big the game was at launch. Games with larger launch audiences retain better, regardless of pricing model. Older games retain worse, which is expected. Free-to-play trended slightly negative after controlling for these factors, but not significantly so.

On review scores, paid games start higher and stay higher, while a subset of free-to-play titles see scores drift downward over time, likely live-service games that introduced unpopular changes post-launch. Neither group showed a statistically significant change from release to present.

The headline takeaway: pricing model alone is a weak predictor of retention. Launch scale and game age matter more, and the free-vs-paid gap is really a short-term vs long-term story rather than one model being strictly better. While this does not align with our expectations in the beginning, our analysis showed there isn't a conclusive " x is better than y " as the context matters.

6 Limitations

Several limitations of this analysis should be acknowledged.

First, review scores for games released before Steam's current review system (2013) may not reflect true launch-window sentiment, as early reviews were retroactively included rather than captured at release. Second, a number of titles in the dataset have switched pricing models over time, most notably games that launched as paid and later transitioned to free-to-play, and this was not accounted for in the group assignments, which may introduce noise into the free-vs-paid comparisons. One such popular example is Counter-Strike 2.

Third, singleplayer versus multiplayer classification was identified as a potentially meaningful covariate for retention but could not be incorporated due to time constraints. Finally, complete player data was obtained for only 47 of the top 50 paid games; given time constraints, missing titles were not replaced, leaving the paid group slightly underrepresented.

Had more we more time, this analysis can be expanded further.
